## 1. Асимптотические нотации `O`, `Ω`, `Θ`. Поиск минимума. Почему `Ω(n)`

`O(g)` — верхняя оценка: $\exists c, n_0$: $0 \le f(n) \le c \cdot g(n)$ при $n \ge n_0$ (функция растёт не быстрее g). `Ω(g)` — нижняя: $0 \le c \cdot g(n) \le f(n)$ (не медленнее g). `Θ(g)` — и верхняя, и нижняя одновременно (тот же порядок роста).
Задача поиска минимума в неотсортированном массиве: нижняя оценка `Ω(n)`, потому что без просмотра любого элемента (в том числе последнего) нельзя доказать, что он не минимальный; корректность требует читать все n. Линейный алгоритм достигает этой границы.

```
int FindMin(int[] a){ // O(n) сравнений, обязаны пройти все
    if(a.Length==0) throw new ArgumentException("empty");
    int m=a[0];
    for(int i=1;i<a.Length;i++) if(a[i]<m) m=a[i];
    return m;
}
```

## 2. Сортировка вставками. Сложности. Инверсия. Подсчёт инверсий за `O(n log n)`

Алгоритм: массив разбит на отсортированную левую часть и неотсортированную правую. Текущий элемент вставляется в отсортированную часть сдвигами вправо. Лучший случай (массив отсортирован) — внутренний while не заходит, сложность `O(n)`. Худший (обратный порядок) и средний — каждую итерацию сдвигаем ~i элементов, суммарно $\sim n^2/2 \Rightarrow$ `O(n^2)`.

Инверсия — пара индексов (i<j) с a[i]>a[j]; их число измеряет “разворот” массива. Подсчёт за `O(n log n)` — модификация merge sort: при слиянии, если берём элемент из правой половины, он образует инверсии со всеми оставшимися слева ( `mid-i+1` ).

```
void InsertionSort(int[] a){
    for(int i=1;i<a.Length;i++){
        int key=a[i], j=i-1;
        while(j>=0 && a[j]>key){ a[j+1]=a[j]; j--; }
        a[j+1]=key;
    }
}
long CountInv(int[] a,int l,int r,int[] tmp){
    if(l>=r) return 0;
    int m=(l+r)/2; long
inv=CountInv(a,l,m,tmp)+CountInv(a,m+1,r,tmp);
    int i=l,j=m+1,k=l;
    while(i<=m && j<=r){
        if(a[i]<=a[j]) tmp[k++]=a[i++];
        else { tmp[k++]=a[j++]; inv += m - i + 1; }
    }
    while(i<=m) tmp[k++]=a[i++]; while(j<=r)
tmp[k++]=a[j++];
    for(int t=l;t<=r;t++) a[t]=tmp[t];
    return inv;
}
```

## 3. Сортировка слиянием. Почему всегда `O(n log n)`. Мастер-теорема. Применение к merge и бинарному поиску

Merge sort: делим пополам, рекурсивно сортируем, линейно сливаем. На уровне рекурсии просматриваем все n элементов ( `O(n)` ), а глубина деления на 2 — `log n`, поэтому суммарно `O(n`

`log n)` независимо от входа (нет “плохих” случаев). Мастер-теорема для `T(n)=aT(n/b)+f(n)` : сравниваем `f(n)` с `n^{log_b a}` . Если f меньше — ответ `Θ(n^{log_b a})` ; если равен — `Θ(n^{log_b a} log n)` ; если больше (и регулярность) — `Θ(f(n))` . Для merge ( `a=2,b=2,f=Θ(n)` ) — второй случай ⇒ `Θ(n log n)` . Для бинарного поиска `T(n)=T(n/2)+O(1)` — первый/второй случай с `a=1,b=2,f=Θ(1)` ⇒ `Θ(log n)` .

```
void MergeSort(int[] a,int l,int r,int[] tmp){
    if(l>=r) return;
    int m=(l+r)/2;
    MergeSort(a,l,m,tmp); MergeSort(a,m+1,r,tmp);
    int i=l,j=m+1,k=l;
    while(i<=m && j<=r) tmp[k++]= a[i]<=a[j]?
a[i++]:a[j++];
    while(i<=m) tmp[k++]=a[i++]; while(j<=r)
tmp[k++]=a[j++];
    Array.Copy(tmp,l,a,l,r-l+1);
}
```

## 4. Очередь с приоритетом. Операции. Бинарная куча (min) и её свойство

Очередь с приоритетом хранит элементы и умеет: `Insert` , `Peek` (минимум), `Extract` (удалить минимум), часто `DecreaseKey` . Min-heap — почти полное бинарное дерево, хранимое в массиве; индексация потомков `2i+1/2i+2` , предка `(i-1)/2` . Свойство: ключ в вершине ≤ ключей в детях, поэтому минимум всегда в корне.

```
class MinHeap{
    private readonly List<int> h=new();
    void SiftUp(int i){ while(i>0){int p=(i-1)/2;
if(h[p]<=h[i]) break; (h[p],h[i])=(h[i],h[p]); i=p;} }
```

```
    void SiftDown(int i){ while(true){int
l=2*i+1,r=l+1,b=i; if(l<h.Count && h[l]<h[b]) b=l;
if(r<h.Count && h[r]<h[b]) b=r; if(b==i) break;
(h[b],h[i])=(h[i],h[b]); i=b;} }
    public void Insert(int x){ h.Add(x);
SiftUp(h.Count-1); }
    public int Peek()=>h[0];
    public int Extract(){ int res=h[0]; h[0]=h[^1];
h.RemoveAt(h.Count-1); if(h.Count>0) SiftDown(0);
return res; }
}
```

## 5. Вставка и извлечение минимума в куче. Почему `O(log n)`. Построение кучи за `O(n)`

Вставка: добавить в конец массива и поднимать вверх, пока свойство не восстановится — длина пути ≤ высота дерева `O(log n)`. Извлечение: заменить корень последним элементом, удалить хвост, опустить вниз — тоже `O(log n)`. Построение кучи из произвольного массива — просеивание вниз всех внутренних узлов от `n/2` к 0; суммарная работа линейна ( `O(n)` ), потому что большинство узлов на низких уровнях.

```
void Build(int[] a){ for(int i=a.Length/2;i>=0;i--)
SiftDown(i); } // heapify O(n)
```

## 6. Пирамидальная сортировка. Доказательство `O(n log n)`

Сначала heapify `O(n)`, затем n раз: обмен корня с последним элементом неотсортированной части, усечение кучи и `SiftDown(0)` (`O(log n)` на шаг). Суммарно `O(n log n)`, доп.память `O(1)` — всё в исходном массиве.

```
void HeapSort(int[] a){
    Build(a);
    for(int end=a.Length-1; end>0; end--){
        (a[0],a[end])=(a[end],a[0]);
        SiftDownRange(0,end); // просеивание с новым
размером кучи
    }
}
```

## 7. Бинарный поиск в отсортированном массиве. Сложность. Нижняя/верхняя граница

Бинарный поиск поддерживает, что ответ лежит в `[l,r)`, делит пополам, отбрасывает половину ⇒ `O(log n)` сравнений. Нижняя граница (`lower_bound`) — первый индекс `>= x`, верхняя (`upper_bound`) — первый `> x`; оба корректны при повторах.

```
int Lower(int[] a,int x){ int l=0,r=a.Length;
while(l<r){int m=(l+r)/2; if(a[m]<x) l=m+1; else r=m;}
return l; }
int Upper(int[] a,int x){ int l=0,r=a.Length;
while(l<r){int m=(l+r)/2; if(a[m]<=x) l=m+1; else r=m;}
return l; }
```

## 8. Идея “бинарного поиска по ответу” + пример

Если предикат `Check(mid)` монотонен по mid (истина сохраняется или, наоборот, ложь сохраняется), можно искать минимальный/максимальный mid бинарно. Пример: максимум изделий при ресурсах — если можно сделать X, то можно и меньше; подбираем hi удвоением, потом бинпоиск.

```
int BinarySearchAnswer(int lo,int hi,Func<int,bool> ok)
{
    while(lo<hi){ int mid=(lo+hi+1)/2; if(ok(mid))
lo=mid; else hi=mid-1; }
    return lo;
}
```

## 9. Рандомизированный QuickSelect. Средняя сложность. Медиана медиан. Плохой фиксированный pivot

QuickSelect: выбираем pivot (случайный), partition, идём только в одну часть. Средняя сложность `O(n)` — математическое ожидание на случайных опорах; худшее `O(n^2)` при систематически плохих pivot (например, всегда первый элемент на отсортированном массиве). Алгоритм “медиана медиан” выбирает хороший pivot детерминированно и гарантирует `O(n)` в худшем случае.

```
int QuickSelect(int[] a,int k,Random rng){
    int l=0,r=a.Length-1;
    while(true){
        int p=rng.Next(l,r+1);
        int pos=Partition(a,l,r,p);
        if(pos==k) return a[pos];
        if(k<pos) r=pos-1; else l=pos+1;
    }
}
```

## 10. Амортизированная сложность; отличие от худшего случая; пример динамического массива

Амортизированная стоимость усредняется по длинной последовательности. Худший случай одной операции может быть большим, но случается редко. Динамический массив: редкое удвоение ёмкости и копирование всех элементов — дорого, но на n вставок суммарно `O(n)` копирований ⇒ средняя цена `O(1)`. Можно доказать агрегатным методом, бухгалтерским (копим “монетки” на resize) и потенциалом (потенциал = свободные слоты).

```
class DynArr<T>{ T[] data=new T[1]; int n=0;
    public void Push(T x){ if(n==data.Length)
Array.Resize(ref data,data.Length*2); data[n++]=x; }
}
```

## 11. Очередь на двух стеках; амортизированный анализ; кольцевой буфер

Очередь: вставки кладём в стек `back`, удаления берём из `front`; если `front` пуст, переливаем в него элементы из `back` (разворачивая порядок). Каждый элемент перекладывается не больше одного раза ⇒ амортизированно `O(1)`. Кольцевой буфер — массив с индексами головы/хвоста по модулю длины, без сдвигов и переполнений.

```
class TwoStackQueue<T>{
    Stack<T> front=new(), back=new();
    public void Enqueue(T x)=>back.Push(x);
    public T Dequeue(){ if(front.Count==0)
while(back.Count>0) front.Push(back.Pop()); return
front.Pop(); }
}
```

## 12. DSU: операции; применения; наив и его недостатки

Вопрос из pdf: “Какие две основные операции поддерживает система непересекающихся множеств? Где она применяется? Какова сложность наивной реализации?”
Операции: `Find` возвращает представителя множества; `Union` объединяет два множества. Применения: MST Крускала, онлайн-объединения компонент, динамическая связность. Наив: родительский массив без эвристик, высота дерева может вырасти до `O(n)`, тогда `Find/Union` работают за линейное время.

```
class DisjointSet
{
    private readonly int[] parent;

    public DisjointSet(int n)
    {
        parent = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    public int Find(int v)
    {
        if (parent[v] == v) return v;
        // без сжатия путей это была бы простая
рекурсия: return Find(parent[v]);
        parent[v] = Find(parent[v]); // добавили сжатие
для ускорения
        return parent[v];
    }

    public void Union(int a, int b)
    {
        a = Find(a);
```

```
        b = Find(b);
        if (a == b) return;
        parent[b] = a; // наивно — всегда подвешиваем
одно под другое
    }
}
```

Без эвристик (сжатия путей/рангов) дерево может стать цепочкой ⇒ `O(n)` на `Find/Union`; эвристики дают амортизированное `O(α(n))` (см. п.13).

## 13. Ранг и сжатие путей; улучшение асимптотики; `α(n)`

Ранг (или размер) — присоединяем меньшее дерево к большему, ограничивая высоту. Сжатие путей делает вершины прямыми детьми корня при `Find`. В сумме амортизированная сложность `O(α(n))`, α — обратная функция Аккермана (очень медленно растёт, практически константа).

```
void Union(int a,int b){
    a=Find(a); b=Find(b); if(a==b) return;
    if(rank[a]<rank[b]) (a,b)=(b,a); parent[b]=a;
if(rank[a]==rank[b]) rank[a]++;
}
```

## 14. Рюкзак 0/1: постановка; рекуррент; оптимизация памяти; обратный порядок

Рекуррент: `dp[i,w]=max(dp[i-1,w], dp[i-1,w-w_i]+c_i)` при `w_i<=w`. Память: можно хранить один массив `dp[w]`, если веса обходить сверху вниз — тогда предмет не используется дважды в одном слое. Сложность `O(n·W)`.

```
int Knapsack01(int[] w,int[] c,int W){
    int[] dp=new int[W+1];
    for(int i=0;i<w.Length;i++)
        for(int weight=W; weight>=w[i]; weight--)
            dp[weight]=Math.Max(dp[weight], dp[weight-
w[i]]+c[i]);
    return dp.Max();
}
```

## 15. Восстановление набора предметов; отличие неограниченного рюкзака

Храним `take[i,w]` или сравниваем `dp[w]` с `dp[w-w_i]+c_i` при проходе назад: если равны, предмет взят, уменьшаем w. В неограниченном рюкзаке внутренний цикл идёт вперёд, предмет можно брать много раз; восстановление должно учитывать, сколько раз взятие улучшало `dp`.

## 16. ДП по подмножествам; пример TSP; ДП по профилю

ДП по подмножествам: состояние — маска, переход — добавление бита. Пример TSP: `dp[mask][v]=min(dp[mask\{v}][u]+dist[u,v])`. ДП по профилю хранит состояние “границы” (занятость клеток в полосе при замощениях/путях), переходы обновляют профиль при добавлении домино/шага.

```
for(int mask=1; mask<(1<<n); mask++)
  for(int v=0; v<n; v++) if((mask&(1<<v))!=0)
    dp[mask][v]=min over u (dp[mask^(1<<v)][u]+w[u,v]);
```

## 17. Дерево отрезков: что это; операции/сложности; хранение

Полное двоичное дерево над массивом: листья — элементы, вершины — агрегаты (min/сумма и т.п.). Хранится в массиве от 1, дети — 2v и 2v+1. Построение `O(n)`, запрос и точечное обновление `O(log n)`.

```
(int mn,int cnt) Combine((int mn,int cnt) A,(int mn,int cnt) B){
    if(A.mn==B.mn) return (A.mn,A.cnt+B.cnt);
    return A.mn<B.mn?A:B;
}
```

## 18. Запрос и обновление в сегдереве; идея отложенных обновлений

Запрос разбивает нужный отрезок на покрывающие узлы, комбинируя ответы; точечное обновление спускается к листу и пересчитывает путь к корню. Ленивые теги: если обновляем диапазон, кладём тег в вершину и раскатываем на детей при обращении, сохраняя `O(log n)` на диапазонных операциях.

```
void Push(int v){ if(lazy[v]!=0){
ApplyChild(2*v,lazy[v]); ApplyChild(2*v+1,lazy[v]);
lazy[v]=0; } }
```

## 19. Дерево Фенвика: операции; преимущества; `i & -i`

Фенвик — компактный массив частичных сумм. `Add(i,delta)`: `i += i & -i`; `Sum(i)`: `i -= i & -i` — обе `O(log n)`. `i & -i` выделяет младший значащий бит, показывая размер блока, который покрывает вершина. Преимущество — простота и компактность по сравнению с сегдеревом (но только префиксы и ассоциативные операции).

```
void Add(int i,int delta){ for(; i<=n; i+= i & -i)
bit[i]+=delta; }
int Sum(int i){ int s=0; for(; i>0; i-= i & -i)
s+=bit[i]; return s; }
```

## 20. Разреженная таблица: задачи, ограничения; предобработка/запрос; почему `O(1)`

Sparse table предназначена для статических idempotent-операций (min/max/gcd), где обновлений нет. Предобработка: `st[k][i]` — результат на отрезке длиной `2^k` от i, строится за `O(n log n)`. Запрос `[l,r]` берёт два перекрывающихся блока длины `2^k` и комбинирует за `O(1)`. Обновления невозможны без перестроения.

```
int Query(int l,int r){ int k=lg[r-l+1]; return
Math.Min(st[k,l], st[k,r-(1<<k)+1]); }
```

## 21. Сбалансированное БДП: что это, зачем; проблема несбалансированного дерева

Сбалансированное БДП удерживает высоту `O(log n)`, обеспечивая логарифмические поиск/вставку/удаление. Несбалансированное дерево при отсортированных вставках вырождается в список с высотой `O(n)`, операции становятся линейными.

## 22. Инвариант AVL; повороты; идея сплей-дерева; подъём к корню

AVL: для каждой вершины `|hL - hR| <= 1`; нарушений исправляем одно-/двойными поворотами, поднимаясь к корню.

Сплей: не хранит баланс, а при доступе поднимает узел к корню (zig, zig-zig, zig-zag), распределяя балансировку по обращениям.

```
Node RotateRight(Node y){ var x=y.Left; y.Left=x.Right;
x.Right=y; FixHeights(y); FixHeights(x); return x; }
```

## 23. AVL vs сплей

AVL — жёсткий баланс, гарантия `O(log n)` на каждую операцию, но больше ребалансировок и хранение высот. Сплей — амортизированное `O(log n)`, быстро обслуживает “горячие” ключи, но отдельные операции могут быть медленнее, высота временно растёт. Выбор зависит от требований к худшему случаю и типу нагрузки.

## 24. Декартово дерево: два свойства

Treap сочетает свойства БДП по ключу (левые ключи меньше, правые больше) и кучи по приоритету (приоритет родителя выше приоритетов детей). Случайные приоритеты дают ожидаемо логарифмическую высоту без сложных инвариантов. В неявном treap ключ заменён позицией/размером поддерева.

## 25. Merge и split в treap; сложность; применение к последовательностям

`Split` режет дерево на два по ключу/позиции за `O(log n)`; `Merge` склеивает, выбирая корень по приоритету, тоже `O(log n)`. Неявный treap позволяет делать вставки, удаления, развороты отрезков через комбинации split+merge.

```
(Node?,Node?) Split(Node? v,int key){
    if(v==null) return (null,null);
```

```
    if(key<=v.key){ var (l,r)=Split(v.l,key); v.l=r;
Update(v); return (l,v); }
    else { var (l,r)=Split(v.r,key); v.r=l; Update(v);
return (v,r); }
}
```

## 26. Сравнение treap с сегдеревом и др.

Treap — простая динамическая структура с ожиданием `O(log n)` на вставку/удаление/разрез. Сегдерево лучше, когда нужны сложные агрегаты и ленивые обновления, но оно не умеет легко резать/склеивать последовательности. Выбор — по набору операций.

## 27. LCA: определение; бинарные подъёмы; предобработка; сложность

LCA(u,v) — самый глубокий общий предок u и v. Бинарные подъёмы: предобработка `up[v][k]` (2^k-й предок) и глубин за `O(n log n)`. Запрос: выравниваем глубины подъёмами, затем поднимаем оба от больших степеней к малым, пока предки не совпадут; их родитель — ответ. Время запроса `O(log n)`.

```
int Lca(int a,int b){
    if(depth[a]<depth[b]) (a,b)=(b,a);
    int diff=depth[a]-depth[b];
    for(int k=LOG-1;k>=0;k--) if(((diff>>k)&1)==1)
a=up[a,k];
    if(a==b) return a;
    for(int k=LOG-1;k>=0;k--) if(up[a,k]!=up[b,k]){
a=up[a,k]; b=up[b,k]; }
    return up[a,0];
}
```

## 28. Сведение LCA к RMQ; эйлеров обход; sparse table; Фарах-Колтон–Бендер

Делаем эйлеров обход дерева, записываем вершины и их глубины. LCA двух вершин — вершина с минимальной глубиной между первыми появлениями ⇒ решается как RMQ (min на отрезке) с помощью sparse table за `O(1)` после `O(n log n)` подготовки. Алгоритм Фарах-Колтона–Бендера оптимизирует память до `O(n)` с тем же `O(1)` временем запроса, используя микро/макроблоки.

## 29. Heavy-Light декомпозиция: понятия, построение, свойства

Вопрос из pdf: “Что такое Heavy-Light декомпозиция? Каковы её ключевые понятия? Опишите шаги построения. Какие свойства дерева используются?”
Ключевые понятия: `sz[v]` — размер поддерева; тяжёлое ребро — к сыну с максимальным поддеревом; лёгкое — остальные; тяжёлый путь — цепочка тяжёлых рёбер. У каждой вершины не более одного тяжёлого сына, каждая вершина лежит ровно на одном тяжёлом пути.
Построение (2 DFS): 1) считаем `sz` и помечаем тяжёлого сына для каждой вершины; 2) раскладываем вершины по путям: входя в тяжёлого сына, сохраняем текущую голову, по лёгким рёбрам начинаем новый путь; вершины одного пути нумеруются подряд и сопоставляются отрезку массива. Свойство: при проходе по лёгкому ребру размер поддерева как минимум вдвое меньше, поэтому на любом пути до корня ≤ `O(log n)` лёгких рёбер ⇒ любой путь разбивается на `O(log n)` тяжёлых отрезков.

```
void DfsSizes(int v,int p){
    parent[v]=p; sz[v]=1; heavy[v]=-1;
```

```
    foreach(var to in g[v]) if(to!=p){
        DfsSizes(to,v); sz[v]+=sz[to];
        if(heavy[v]==-1 || sz[to]>sz[heavy[v]])
heavy[v]=to;
    }
}
void Decompose(int v,int head){
    headOf[v]=head; pos[v]=timer++;
    if(heavy[v]!=-1) Decompose(heavy[v],head);
    foreach(var to in g[v]) if(to!=parent[v] &&
to!=heavy[v]) Decompose(to,to);
}
```

## 30. Как HLD используется для запросов на путях

Вопрос из pdf: “Как HLD используется для выполнения запросов на путях в дереве? Приведите пример.”
Путь `u-v` разбиваем на `O(log n)` отрезков массива: пока головы цепей разные, берём цепь с глубокой головой, добавляем запрос по отрезку `[head, u]`, поднимаем вершину к родителю головы (лёгкое ребро). Когда вершины в одной цепи — один запрос по их позициям. Итоговую операцию (сумма/мин/макс) агрегируем. С сегдеревом на массиве сложность `O(log^2 n)` (или `O(log n)` с idempotent RMQ). Пример из pdf: сумма на пути — строим сегдерево поверх позиций после HLD, разбиваем путь на отрезки, суммируем ответы.

## 31. Центроидная декомпозиция: центроид; построение; сравнение с HLD

Вопрос из pdf: “Что такое центроидная декомпозиция? Как находится центроид? Опишите алгоритм построения и сравните с HLD.”

Центроид — вершина, при удалении которой каждая компонента имеет размер ≤ n/2 (n — размер текущего дерева/компоненты).
Нахождение: DFS для размеров поддеревьев, затем спускаемся в ребёнка, если его поддерево > n/2, пока не найдём подходящую вершину.
Построение: 1) ищем центроид компоненты, добавляем его в дерево центроидов; 2) “удаляем” его, рекурсивно строим декомпозицию для получившихся поддеревьев; 3) получаем дерево центроидов (родитель — центроид верхнего уровня).
Используется для задач расстояний/пар с разложением по слоям центроидов.
Сравнение с HLD: HLD режет дерево на тяжёлые пути и отвечает запросы на путях (u–v) через `O(log n)` отрезков; центроидная декомпозиция режет по центрам, удобна для задач, где важно расстояние/покрытие от набора вершин. Структура — дерево центроидов, а не интервальная раскладка пути.

## 32. DFS: алгоритм, сложность; поиск цикла в неориентированном/ориентированном графе

DFS красит вершину (белый→серый→чёрный), рекурсивно посещает непосещённых соседей. Сложность `O(n+m)`. Цикл: в неориентированном — ребро к серой вершине, не являющейся родителем; в ориентированном — ребро к вершине в стеке (серой).

```
void Dfs(int v,int p){
    color[v]=1;
    foreach(var to in g[v]){
        if(color[to]==0) Dfs(to,v);
        else if(color[to]==1 && to!=p) hasCycle=true;
    }
```

```
    color[v]=2;
}
```

## 33. Топологическая сортировка: Кана и DFS; применения

Топсорт упорядочивает DAG так, чтобы все рёбра шли слева направо. Кана: берём вершины с входной степенью 0, удаляем их и их рёбра; повторяем до опустошения или обнаружения цикла. DFS-вариант: записываем вершины в порядке выхода (постфикс), разворачиваем список. Применения — планирование, порядок зависимостей, компиляция.

```
List<int> TopoKahn(){
    var q=new Queue<int>(); for(int v=0;v<n;v++)
if(indeg[v]==0) q.Enqueue(v);
    var order=new List<int>();
    while(q.Count>0){ int v=q.Dequeue(); order.Add(v);
foreach(var to in g[v]) if(--indeg[to]==0)
q.Enqueue(to); }
    return order;
}
```

## 34. Компоненты сильной связности; алгоритм Косарайю; конденсация

КСС — максимальные подграфы, где вершины достижимы друг из друга. Косарайю: 1) DFS на исходном графе, записываем вершины по времени выхода; 2) DFS на транспонированном графе в обратном порядке списка — каждая волна даёт компоненту. Конденсация — DAG компонент, где дуги отражают связи между ними.

```
void Dfs1(int v){ used[v]=true; foreach(var to in g[v])
if(!used[to]) Dfs1(to); order.Add(v); }
void Dfs2(int v,int c){ comp[v]=c; foreach(var to in
gr[v]) if(comp[to]==-1) Dfs2(to,c); }
```

## 35. Мосты и точки; алгоритм; Эйлеров цикл, условия и поиск

Мост — ребро, удаление которого увеличивает число компонент; точка сочленения — вершина с тем же свойством. Используем tin/low: мост, если `low[to] > tin[v]`; точка — если есть ребёнок с `low[child] >= tin[v]` (для корня — ≥2 детей). Эйлеров цикл в неориентированном графе существует, если граф связен по рёбрам и все степени чётные; строится алгоритмом Хиерхольцера, удаляя рёбра при обходе.

```
void Dfs(int v,int p){
    tin[v]=low[v]=timer++;
    foreach(var to in g[v]){
        if(to==p) continue;
        if(tin[to]!=-1) low[v]=Math.Min(low[v],
tin[to]);
        else{ Dfs(to,v); low[v]=Math.Min(low[v],
low[to]); if(low[to]>tin[v]) bridges.Add((v,to));
if(p!=-1 && low[to]>=tin[v]) artic.Add(v); }
    }
}
```

## 36. MST: определение; Крускал и Прим; сравнение

MST — остов минимального веса. Крускал: сортируем рёбра, берём лёгкие, если концы в разных компонентах (DSU) — `O(m`

`log m)` . Прим: растим дерево, выбирая минимальное ребро наружу из построенной части (обычно через кучу) — `O(m log n)` или `O(m + n log n)` для разреженных графов. Крускал прост для списков рёбер, Прим удобен, когда нужно расти из стартовой вершины.

```
edges.Sort((x,y)=>x.w.CompareTo(y.w));
foreach(var e in edges) if(Find(e.u)!=Find(e.v)){
Union(e.u,e.v); mst+=e.w; }
```

## 37. Алгоритмы кратчайших путей; условия; описания

BFS — невзвешенный граф, `O(n+m)` . Дейкстра — неотрицательные веса, `O(m log n)` с приоритетной очередью. Беллман–Форд — работает с отрицательными рёбрами, ловит отрицательные циклы, `O(nm)` . Флойд–Уоршелл — все пары на плотных графах, `O(n^3)` . Джонсон — перенормировка потенциалами + Дейкстра для всех пар в разреженных графах без отрицательных циклов.

```
var dist=Enumerable.Repeat(long.MaxValue,n).ToArray();
dist[s]=0;
var pq=new PriorityQueue<(int v,long d),long>();
pq.Enqueue((s,0),0);
while(pq.Count>0){ var (v,d)=pq.Dequeue();
if(d!=dist[v]) continue; foreach(var (to,w) in g[v])
if(d+w<dist[to]){ dist[to]=d+w;
pq.Enqueue((to,dist[to]), dist[to]); } }
```

## 38. Хеш-таблица: операции; цепочки и открытая адресация; коэффициент заполнения и ре-хеш

Хеш-таблица переводит ключ в индекс. Метод цепочек — в бакете хранится список пар; открытая адресация — пробная последовательность до пустой/нужной ячейки. Операции в среднем `O(1)` при контроле коэффициента заполнения; при переполнении делаем rehash (увеличиваем размер, перераспределяем).

```
int Index(string key)=> (key.GetHashCode() &
int.MaxValue) % buckets.Length;
```

## 39. Сравнение цепочек и открытой адресации

Цепочки: легко удалять, устойчивы к кластеризации, гибко хранят разные количества элементов; минус — память на списки и худшая кеш-локальность. Открытая адресация: компактно и кеш-дружелюбно, но чувствительно к заполнению, удаление требует “маркеров” или перестроения, возможна кластеризация. Выбор — по ограничению памяти и профилю операций.

## 40. Идеальное хеширование; двухуровневая схема; кукушкин хеш

Идеальное хеширование — отсутствие коллизий на фиксированном наборе: первый уровень распределяет элементы, второй для каждого бакета подбирает хеш без коллизий. Кукушкин хеш: два хеша/две позиции на элемент, при вставке может “выселять” другие по их альтернативным позициям; при цикле — перестройка. Хорошо работает при умеренном заполнении.

## 41. Фильтр Блума: операции; плюсы/минусы; вероятность ложноположительного

Битовый массив + k хешей. `Add` ставит k бит, `Query` проверяет, что все они 1 — ложноположительные возможны, ложноотрицательные нет (при корректной реализации). Вероятность FP ≈ `(1 - e^{-(k n / m)})^k`; уменьшается при большем m и оптимальном k. Плюсы — компактность и скорость; минусы — нет удаления (кроме счётчиков) и FP.

```
void Add(string s){ foreach(var h in Hashes(s))
bits[h%M]=true; }
bool MightContain(string s){ return
Hashes(s).All(h=>bits[h%M]); }
```

## 42. Класс и объект; связь; элементы класса

Класс описывает поля/свойства/методы/события/конструкторы — абстракция сущности. Объект — экземпляр класса с конкретным состоянием. Связь: класс — шаблон, объект — конкретизация; через класс создаём множество однотипных объектов.

```
class Point{ public int X{get;set;} public int
Y{get;set;} public Point(int x,int y){X=x;Y=y;} }
```

## 43. Конструктор; модификаторы доступа; инкапсуляция; свойства (в т.ч. авто)

Конструктор инициализирует объект. Модификаторы (public/private/protected/internal) задают видимость. Инкапсуляция — скрываем данные, открываем управляемый интерфейс (свойства/методы) для поддержания инвариантов. Свойства могут содержать логику get/set или быть авто (компилятор создаёт поле).

```
class Counter{ private int val; public int Value=>val;
public void Inc()=>val++; }
```

## 44. Статические члены и конструктор; `this`

Статические члены принадлежат типу (один экземпляр на класс), статический конструктор выполняется один раз перед первым использованием. `this` ссылается на текущий объект, позволяет различать поля и параметры и передавать сам объект.

```
class Sample{ static int Created; public Sample()
{Created++;} public void
Show()=>Console.WriteLine(this.ToString()); }
```

## 45. Наследование; синтаксис; какие члены наследуются; sealed; ограничения

`class Child : Parent` — наследует открытые/защищённые члены (учитывая модификаторы), может переопределять виртуальные. Запечатанный ( `sealed` ) класс нельзя расширять. В C# — одиночное наследование классов.

```
class Animal{ public virtual void Sound(){} }
class Cat:Animal{ public override void
Sound()=>Console.WriteLine("Meow"); }
```

## 46. Когда вызывать конструктор базового класса; `base`

Если базовый требует параметров или инициализации, производный вызывает его через список инициализации `:`

`base(args)`. `base.Member` используют, чтобы явно обратиться к базовой реализации или конструктору.

```
class B{ public B(int x){} }
class D:B{ public D():base(42){} }
```

## 47. Полиморфизм; `virtual`; `override` vs `new`; абстрактный класс/метод; экземпляр абстрактного класса

Полиморфизм — единый интерфейс, разные реализации. `virtual` разрешает переопределение. `override` заменяет базовую реализацию с полиморфизмом; `new` скрывает, но при обращении через базовый тип сработает базовый метод. Абстрактный класс/метод задаёт обязательный контракт; абстрактный класс нельзя инстанцировать.

```
abstract class Shape{ public abstract double Area(); }
class Square:Shape{ public double A; public override
double Area()=>A*A; }
```

## 48. Перегрузка методов; анонимные типы и применение

Перегрузка — несколько методов с одним именем и разными параметрами, компилятор выбирает по сигнатуре. Анонимные типы (`new { Name="x", Age=1 }`) создаются на лету, часто в LINQ для временных проекций; свойства только для чтения.

```
int Sum(int a,int b)=>a+b; int Sum(int a,int b,int
c)=>a+b+c;
```

## 49. Интерфейс vs абстрактный класс; сколько интерфейсов; наследование интерфейсов; реализация; модификаторы

Интерфейс — чистый контракт (в классическом виде без реализации/состояния); абстрактный класс может содержать поля и готовые методы. Класс может реализовать множество интерфейсов. Интерфейсы могут наследоваться. Все члены интерфейса публичные по умолчанию, реализовать нужно все (кроме default-методов в новых версиях).

```
interface IFly{ void Fly(); } class Bird:IFly{ public
void Fly(){} }
```

## 50. Делегат; многоадресный; событие vs делегат; анонимный метод; шаблон `EventArgs`

Делегат — типизированная ссылка на метод; многоадресный хранит список методов. Событие — обёртка над делегатом, ограничивающая внешние вызовы (только подписка/отписка). Анонимный метод/лямбда задаёт обработчик inline. Шаблон событий: `EventHandler(object sender, EventArgs e)`, свои данные — наследование от `EventArgs`.

```
public delegate void Handler(string msg);
public event Handler OnEvent;
public void Raise()=>OnEvent?.Invoke("hi");
```

## 51. Исключения: зачем; `try-catch-finally`; когда выполняется `finally`; `throw`; свои исключения

*(Вопрос отсутствует в лекционных конспектах; краткий ответ приведён для полноты.)*

Исключения сигнализируют об ошибках. `try-catch-finally` перехватывает и даёт гарантированный блок `finally`; `finally` не выполняется только при аварийном завершении процесса. `throw` генерирует исключение; собственные исключения наследуют от `Exception`, добавляя контекст.

```
try{ Do(); } catch(Exception ex){
Console.WriteLine(ex.Message); } finally{ Clean(); }
```